

Query Data with DynamoDB

Melvin green

```
>
>
> aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"201"}}'
>
> ^C
nextworksampldata $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"201"}}'
{
  "Item": {
    "Title": {
      "S": "AWS Relational vs Non Relational Databases in 6 minutes"
    },
    "Price": {
      "N": "0"
    },
    "VideoType": {
      "S": "Educational"
    },
    "ContentType": {
      "S": "Video"
    },
    "Services": {
      "L": [
        {
          "S": "Amazon RDS"
        },
        {
          "S": "Amazon DynamoDB"
        }
      ]
    },
    "Id": {
      "N": "201"
    },
    "URL": {
      "S": "https://youtu.be/SzvdEf9y0eA"
    }
  }
}
nextworksampldata $
nextworksampldata $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"101"}}' \
> --consistent-read \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 1.0
  }
}
nextworksampldata $
```

Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed, serverless NoSQL database service provided by AWS. It stores data as items in tables, where each item is identified by a primary key (a partition key,

optionally combined with a sort key), rather than using the rows/columns/joins model of traditional relational databases.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to create and manage tables for storing forum-related data, including ContentCatalog, Forum, Post, and Comment. I loaded sample data into these tables, ran basic queries and scans through both the AWS Console and CloudShell using the AWS CLI, and used a transaction to update two related tables, the Comment and Forum tables, in a single operation to keep the comment count accurate whenever a new comment was added.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is that a query on a non-key attribute, like PostedBy, wouldn't work without also specifying a partition key, since I initially assumed I could filter on any attribute directly through the Query section.

This project took me...

This project took me 1 hour to complete and document.

Querying DynamoDB Tables

A partition key is an attribute in a DynamoDB table whose value DynamoDB hashes to determine which physical partition (storage segment) an item is stored in. It's the primary way DynamoDB distributes and locates data across its underlying storage, and every Query must specify an exact partition key value.

A sort key is a secondary key used alongside the partition key. DynamoDB first uses the partition key to locate the correct partition, then uses the sort key to order and narrow down items within that partition. Together, partition key + sort key form a unique composite primary key, letting me store multiple items under the same partition key while still being able to query a specific item or a range of items.

Comment

☒ Autopreview [View table details](#)

▼ Scan, query or search items [Info](#)

☐ Scan

☒ Query

☐ Search

Select a table or index

Table - Comment ▼

Select attribute projection

All attributes ▼

Partition key

Attribute

Id

Value

I have a question/Just Complete Project #7 Dependencies and CodeArtifacts

Sort key - optional

Attribute

CommentDateTime

Value

Greater than ▼ 2024-09-01

☐ Sort descending

► Filters - optional

Run

Reset

Limits of Using DynamoDB

I ran into an error when I queried using PostedBy as the attribute name and User Abdulrahman as the value in the expanded filter section of the Query section. This happened because I didn't use a partition key when querying the items.

and base it off the sentence I gave you Insights we could extract from our Comment table include comments on a specific thread sorted by time, how many comments a thread has, the most recent comment on a thread, and comments within a date range for a known thread, since the partition key Id and sort key CommentDateTime let us query efficiently within a single thread. Insights we can't easily extract from the Comment table include all comments posted by a specific user, the total number of comments across the whole table, and which threads have the most engagement, since none of these start from a known Id and would require a full table Scan or a Global Secondary Index, such as one on PostedBy, to retrieve efficiently.

The screenshot shows the AWS DynamoDB console interface for configuring a query on the 'Comment' table. The 'Scan, query or search items' section has the 'Query' option selected. The 'Select a table or index' dropdown is set to 'Table - Comment', and the 'Select attribute projection' dropdown is set to 'All attributes'. In the 'Partition key' section, the 'Attribute' is 'Id' and the 'Value' field is empty, which has triggered a red error message: 'The partition key filter cannot be empty.' Below this, the 'Sort key - optional' section is configured with 'Attribute' as 'CommentDateTime', 'Value' as 'Equal to', and a 'Sort descending' checkbox. At the bottom, the 'Filters - optional' section contains a single filter with 'Attribute name' as 'PostedBy', 'Condition' as 'Equal to', 'Type' as 'String', and 'Value' as 'User Abdulrahman'. A 'Remove' button is next to the filter.

Attribute name	Condition	Type	Value	Action
PostedBy	Equal to	String	User Abdulrahman	Remove

Running Queries with CLI

A query I ran in CloudShell was `aws dynamodb get-item --table-name ContentCatalog --key '{"Id":{"N":"101"}}' --consistent-read --projectionexpression "Title, ContentType, Services" --return-consumed-capacity TOTAL`. This query gets the item from the ContentCatalog table with an Id of 101, using a strongly consistent read, and returns only the Title, ContentType, and Services attributes for that item.

Now lets turn that into a paragrph The `aws dynamodb get-item` command is used to retrieve a single item from a DynamoDB table by its primary key. The `-table-name ContentCatalog` option tells DynamoDB which table to look in, in this case ContentCatalog, while the `--key '{"Id":{"N":"201"}}'` option specifies the primary key value of the item to retrieve, where `Id` is the attribute name, `"N"` indicates the value is a Number type, and `"201"` is the actual key value being looked up. Other options can be added to customize the query further, such as `-consistent-read`, which forces a strongly consistent read to guarantee the most up to date data, `--projection-expression`, which limits the response to only specific attributes instead of the whole item, and `--return-consumed-capacity TOTAL`, which includes the read capacity units consumed by the request, useful for monitoring cost and throughput.

```

>
> aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"201"}}'
>
> ^C
nextworksampdata $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"201"}}'
{
  "Item": {
    "Title": {
      "S": "AWS Relational vs Non Relational Databases in 6 minutes"
    },
    "Price": {
      "N": "0"
    },
    "VideoType": {
      "S": "Educational"
    },
    "ContentType": {
      "S": "Video"
    },
    "Services": {
      "L": [
        {
          "S": "Amazon RDS"
        },
        {
          "S": "Amazon DynamoDB"
        }
      ]
    },
    "Id": {
      "N": "201"
    },
    "URL": {
      "S": "https://youtu.be/SzvdEf9y0eA"
    }
  }
}
nextworksampdata $
nextworksampdata $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"101"}}' \
> --consistent-read \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 1.0
  }
}
nextworksampdata $

```

Transactions

A transaction is a group of operations that are executed together, so that either all of them succeed or none of them do. This ensures that related updates across multiple tables, like adding a new comment and incrementing the forum's comment count, stay consistent and accurate, since DynamoDB won't apply just one of the changes without the other.

I ran a transaction that performed two operations at once. The first was a Put operation on the Comment table, adding a new item with Id set to "Events/Do a Project Together - NextWork Study Session", a CommentDateTime, a Comment of "Excited to attend!", and PostedBy set to "User Connor". The second was an Update operation on the Forum table, targeting the item with Name "Events" and using the UpdateExpression "ADD Comments :inc" to increment its Comments count by 1. Because both operations were part of the

same transaction, DynamoDB ensured that the new comment was added and the forum's comment count was updated together, keeping the two tables consistent.

```
}
nextworksampladata $ aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[
> {
>   "Put": {
>     "TableName": "Comment",
>     "Item": {
>       "Id": {"S": "Events/Do a Project Together - NextWork Study Session"},
>       "CommentDateTime": {"S": "2024-9-27T17:47:30Z"},
>       "Comment": {"S": "Excited to attend!"},
>       "PostedBy": {"S": "User Connor"}
>     }
>   },
>   "Update": {
>     "TableName": "Forum",
>     "Key": {"Name": {"S": "Events"}},
>     "UpdateExpression": "ADD Comments:inc",
>     "ExpressionAttributeValues": {":inc": {"N": "1"} }
>   }
> ]'
nextworksampladata $
nextworksampladata $
```